

SD Basic - Restricted Commands

The statements and functions an ordinary program cannot compile, and what the compiler says when it tries.

Nothing on this page is available to an application: every name here needs a program compiled with `$internal`, which in turn needs an administrator in the `SDSYS` account. They are listed because they exist, because they appear in SD's own source, and because the errors they produce name something other than the real cause.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

Italics mark something you supply, **bold** a word typed as it stands, and braces an optional part.

This page is generated from `BCOMP`'s **own tables** by `tools/mksyntax.py`, the same script that builds the User set's syntax card, and the two **partition** the roster: every name the compiler accepts is on exactly one of them, and the script refuses to write either page if that stops being true.

The three kinds, and how each one fails

(restricted)	a statement in <code>BCOMP</code> 's <code>restricted.statements</code> . An ordinary account gets "Unrecognised statement" , which at least names the right line
(internal)	a function in <code>BCOMP</code> 's <code>int.intrinsics</code> . The message names something else entirely : an unknown function is read as a matrix reference, so the complaint is "Matrix X is not referenced in a DIM statement" — reported at the last line of the program , nowhere near the call. With three or more arguments it is <i>"Right bracket not found where expected"</i> instead, because a matrix takes at most two subscripts
(no such thing)	in a table with no opcode behind it. It compiles for nobody — <code>\$internal</code> does not help

THE SECOND ROW IS THE ONE TO REMEMBER. If a function you are certain exists produces a complaint about a `dim` statement you never wrote, it does exist, and this account may not call it.

`$internal` **needs both halves**. `BCOMP` tests `kernel(K$INTERNAL, -1)` and `kernel(K$ADMINISTRATOR, -1)` — internal mode alone was enough until 13 Aug 2026, and it was not safe: internal programs are the only ones that may set the administrator flag, and `sd -internal` is not itself gated, so any account could have compiled a three-line program that granted itself administrator rights. That was demonstrated, not theorised.

A

<code>abort.cause</code>	<code>abort.cause()</code> (internal)
<code>add</code>	<code>add</code> <code>n</code> <code>to</code> <code>variable</code> (restricted)
<code>akclear</code>	<code>akclear</code> <code>file.var, index</code> <code>then</code> ... <code>else</code> ... (restricted)
<code>akdelete</code>	<code>akdelete</code> <code>file.var, index, key</code> <code>then</code> ... <code>else</code> ... (restricted)
<code>akenable</code>	<code>akenable</code> <code>file.var, index, state</code> (restricted)
<code>akmap</code>	<code>akmap</code> (<code>a, b</code>) (internal)
<code>akread</code>	<code>akread</code> <code>var</code> <code>from</code> <code>file.var, index, key</code> <code>then</code> ... <code>else</code> ... (restricted)
<code>akrelease</code>	<code>akrelease</code> <code>file.var, index, key</code> (restricted)
<code>akwrite</code>	<code>akwrite</code> <code>var</code> <code>to</code> <code>file.var, index, key</code> <code>then</code> ... <code>else</code> ... (restricted)
<code>analyse</code>	<code>analyse</code> (<code>a</code>) (internal)

B

<code>break.count</code>	<code>break.count()</code> (internal)
<code>breakpoint</code>	<code>breakpoint</code> <code>action, qualifier</code> (restricted)
<code>btree</code>	<code>btree</code> (<code>file.var, matrix</code>) — <code>matrix</code> is a <code>dim</code> med one-dimensional name (internal)

C

<code>callv</code>	<code>callv</code> <code>name</code> (<code>arg, ...</code>) (restricted)
<code>changed</code>	<code>changed</code> (<code>a</code>) (internal)
<code>como</code>	<code>como on</code> <code>name</code> · <code>como off</code> · <code>como</code> <code>name</code> { <code>on error</code> ... } (restricted)

<code>configure.file</code>	<code>configure.file</code> <i>file.var, key, value</i> <code>then</code> ... <code>else</code> ... (restricted)
-----------------------------	---

<code>create.ak</code>	<code>create.ak</code> <i>file.var, index, dict.rec</i> <code>then</code> ... <code>else</code> ... (restricted)
------------------------	---

D

<code>debug.info</code>	<code>debug.info</code> (<i>a, b</i>) (internal)
-------------------------	---

<code>debug.off</code>	<code>debug.off</code> (restricted)
------------------------	--

<code>debug.on</code>	<code>debug.on</code> (restricted)
-----------------------	---

<code>debug.set</code>	<code>debug.set</code> <i>var {, qualifier}</i> <code>to</code> <i>value</i> (restricted)
------------------------	--

<code>delete.ak</code>	<code>delete.ak</code> <i>file.var, index</i> <code>then</code> ... <code>else</code> ... (restricted)
------------------------	---

<code>delete.common</code>	<code>delete.common</code> <i>name</i> (restricted)
----------------------------	--

E

<code>errmsg</code>	(no such thing) — in the statement table with no opcode behind it
---------------------	--

<code>events</code>	<code>events</code> (<i>a, b</i>) (internal)
---------------------	---

<code>expand.hf</code>	<code>expand.hf</code> (<i>a, b, c</i>) (internal)
------------------------	---

F

<code>fcontrol</code>	<code>fcontrol</code> (<i>a, b, c</i>) (internal)
-----------------------	--

<code>find</code>	<code>find</code> <i>x</i> <code>in</code> <i>arr {, occ}</i> <code>setting</code> <i>f {, v {, sv}}</i> <code>then</code> ... <code>else</code> ... (internal)
-------------------	--

<code>formcsv</code>	<code>formcsv</code> (<i>a</i>) (internal)
----------------------	---

G

<code>getlocks</code>	<code>getlocks(<i>a</i>, <i>b</i>)</code> <i>(internal)</i>
-----------------------	---

<code>grpstat</code>	<code>grpstat(<i>a</i>, <i>b</i>)</code> <i>(internal)</i>
----------------------	--

I

<code>is.subr</code>	<code>is.subr(<i>a</i>)</code> <i>(internal)</i>
----------------------	--

<code>ismv</code>	<code>ismv(<i>a</i>)</code> <i>(internal)</i>
-------------------	---

K

<code>kernel</code>	<code>kernel(<i>a</i>, <i>b</i>)</code> <i>(internal)</i>
---------------------	---

<code>keyboard.input</code>	<code>keyboard.input</code> <i>string (restricted)</i>
-----------------------------	--

L

<code>list.common</code>	<code>list.common()</code> <i>(internal)</i>
--------------------------	--

<code>load.object</code>	<code>load.object(<i>a</i>)</code> <i>(internal)</i>
--------------------------	--

<code>loaded</code>	<code>loaded(<i>a</i>)</code> <i>(internal)</i>
---------------------	---

<code>login</code>	<code>login(<i>a</i>, <i>b</i>)</code> <i>(internal)</i>
--------------------	--

<code>login.port</code>	<code>login.port(<i>a</i>, <i>b</i>)</code> <i>(internal)</i>
-------------------------	---

<code>logout</code>	<code>logout(<i>a</i>, <i>b</i>)</code> <i>(internal)</i>
---------------------	---

M

<code>modify</code>	<code>modify</code> <i>file.var, id</i> (restricted)
---------------------	---

O

<code>option</code>	<code>option(</code> <i>a</i> <code>)</code> (internal)
---------------------	--

<code>ospath</code>	<code>ospath(</code> <i>a, b</i> <code>)</code> (internal)
---------------------	---

<code>osrename</code>	<code>osrename(</code> <i>a, b</i> <code>)</code> (internal)
-----------------------	---

P

<code>pconfig</code>	<code>pconfig(</code> <i>a, b</i> <code>)</code> (internal)
----------------------	--

<code>phantom</code>	<code>phantom()</code> (internal)
----------------------	--

<code>prompt</code>	<code>prompt</code> <i>character</i> (internal)
---------------------	--

<code>pterm</code>	<code>pterm(</code> <i>a, b</i> <code>)</code> (internal)
--------------------	--

<code>pwcrypt</code>	<code>pwcrypt(</code> <i>a</i> <code>)</code> (internal)
----------------------	---

Q

<code>quit</code>	<code>quit</code> (restricted)
-------------------	---------------------------------------

R

<code>readpkt</code>	<code>readpkt()</code> (internal)
<code>release.lock</code>	<code>release.lock</code> <i>n</i> (restricted)
<code>remove.token</code>	<code>remove.token</code> (restricted)
<code>removef</code>	<code>removef(<i>string</i> {, <i>occurrence</i>})</code> (internal)
<code>reset.modes</code>	<code>reset.modes</code> <i>mask</i> (restricted)
<code>rewind</code>	<code>rewind</code> <i>file.var</i> <code>then</code> ... <code>else</code> ... (restricted)
<code>run</code>	<code>run</code> { <i>file</i> } <i>record</i> { <i>options</i> } (restricted)

S

<code>scan</code>	<code>scan(<i>a</i> {, <i>matrix</i>})</code> — <i>matrix</i> is a <code>dim</code> med one-dimensional name (internal)
<code>sdext</code>	<code>sdext(<i>a</i>, <i>b</i>, <i>c</i>)</code> (internal)
<code>set.modes</code>	<code>set.modes</code> <i>mask</i> (restricted)
<code>set.status</code>	<code>set.status</code> <i>n</i> (restricted)
<code>set.trigger</code>	<code>set.trigger</code> <i>file.var</i> , <i>mode</i> , <i>program</i> {, <i>modes</i> } (restricted)
<code>set.unassigned</code>	<code>set.unassigned</code> <i>var</i> (restricted)
<code>sortadd</code>	<code>sortadd</code> <i>key</i> , <i>data</i> (restricted)
<code>sortclear</code>	<code>sortclear</code> (restricted)
<code>sortdata</code>	<code>sortdata()</code> (internal)
<code>sortinit</code>	<code>sortinit</code> <i>keys</i> , <i>memory</i> (restricted)
<code>sortnext</code>	<code>sortnext(<i>matrix</i>)</code> — <i>matrix</i> is a <code>dim</code> med one-dimensional name (internal)

T

`testlock`

`testlock(a)` *(internal)*

U

`unload.object`

`unload.object` *class (restricted)*

V

`varset`

`varset` *var, value (restricted)*

W

`watch`

`watch` *var (restricted)*

`writepkt`

`writepkt` *data (restricted)*

See also

Operating System Access covers the `sh` and `!` verbs, which are the other thing an ordinary account cannot reach. Encryption and the SDEXT interface explains why `sdext` is on this page and what it costs an application.

The User set's *SD Basic - Syntax* card carries everything an application may use, and the two partition the roster between them.